Booking.com Database Management System

Ahmed Abdelmoula
220303923
Bilgisayar Müh/Comp Eng

January 2026

Problem 1: Unclear Cancellation Policies

The Problem: Guests are often confused about whether they can get a refund if they cancel, leading to disputes and lost revenue for hotels.

The Solution: The system utilizes a dedicated `Policy` table. This allows every hotel to define specific rules (e.g., "RefundPercentage", "DeadlineHours", and "EnforcementLevel"), ensuring that cancellation terms are transparent and enforced automatically by the system.

Problem 2: Lack of Customer Loyalty Rewards

The Problem: Frequent travelers often feel unappreciated and switch to competitors if they don't receive perks for their loyalty.

The Solution: The project implements a `GeniusTier` system linked to the `Customer` table. This tracks the user's level (e.g., Bronze, Silver, Gold) and automatically applies the correct DiscountRate to their bookings, encouraging repeat business.

Problem 3: Misleading Facility Information

The Problem: Guests often arrive at a hotel expecting a pool or gym, only to find it under construction or non-existent.

The Solution: The `Hotel_Facilities` and `Facility` tables manage amenities with specific details like "OperationalHours" and "CostType" (Surcharge vs. Complimentary). This ensures accurate, granular information is displayed to the user before they book.

Problem 4: Managing Seasonal Demand

The Problem: Hotels struggle to fill rooms during low seasons or want to offer special deals for holidays without changing base prices manually every day.

The Solution: The `Promotion` table allows hotels to create time-bound marketing campaigns defined by `StartDate` and `EndDate`. The database validates these dates to ensure discounts are only applied during the valid promotional period.

Problem 5: Inaccurate Room Occupancy

The Problem: A family of four might accidentally book a room designed for two people, leading to compliance issues and discomfort upon arrival.

The Solution: The `RoomType` table includes a `MaxOccupancy` constraint. The database enforces safety standards by ensuring that the number of guests does not exceed the room's physical capacity.

Problem 6: Housekeeping & Room Readiness

The Problem: Front desk staff often assign rooms to new guests before they have been cleaned, resulting in a poor customer experience.

The Solution: The `Room` table includes an `IsCleaned` (BIT) status flag. This allows the system to filter available rooms and prevents the check-in of a guest into a dirty room until housekeeping updates the status.

Problem 7: Trust & Quality Assurance

The Problem: Users find it hard to trust hotels based on photos alone and fear poor service or hygiene.

The Solution: The `Review` table captures specific metrics like `CleanlinessScore`, `StaffScore`, and `ComfortScore`. By calculating an average score, the system provides future guests with data-driven social proof of the hotel's quality.

Problem 8: Payment Failures & Security

The Problem: Keeping track of which reservations are paid, pending, or failed is chaotic without a centralized ledger.

The Solution: The `Payment` table records transaction details, including `PaymentStatus` ('Success' or 'Failed') and masked card data. This ensures financial records are accurate and helps verify bookings before confirmation.

Problem 9: International User Experience

The Problem: A global platform must serve users in their native language to increase conversion rates and reduce support calls.

The Solution: The `Language` table and the `PreferredLangCode` column in the `Customer` table allow the system to store user preferences (like Text Direction 'RTL' or 'LTR'), delivering a personalized interface for international guests.

Problem 10: Tax Calculation Errors

The Problem: Different cities and countries have different tax laws, and calculating the final price manually is prone to human error.

The Solution: The `Hotel` table stores a specific `TaxPercentage` for each property. This allows the system to automatically calculate the final booking cost (`TotalCost`) by applying the correct local tax rate to the base price.